



۱. مقدمه و تعریف تمرین

لینوس اخیراً در حال نصب یک پکیج پیچیده به نام BadPG روی سیستم خود بود. در میانه راه، نصب یکی از ماژول‌ها با خطا مواجه شد؛ اما از آنجا که برنامه نصب‌کننده به درستی طراحی نشده بود، تغییرات و ماژول‌های نصب‌شده‌ی قبلی به حالت اول برنگشتند و لینوس مجبور شد ساعت‌ها وقت بگذارد تا سیستم را به صورت دستی پاکسازی کند! این تجربه تلخ، الهام‌بخش او شد تا به کمک دانشی که در درس برنامه‌نویسی پیشرفته کسب کرده است و با استفاده از مفاهیم برنامه‌نویسی شی‌گرا - از جمله کپسوله‌سازی، مدیریت چرخه حیات اشیاء، تخصیص پویای حافظه، وراثت و چندریختی - یک نصب‌کننده‌ی بی‌نقص بسازد.

ساختار مؤلفه‌ها

در این سیستم، هر چیزی که قابل نصب باشد یک **مؤلفه** (component) نامیده می‌شود که همواره در یکی از وضعیت‌های **PENDING**، **INSTALLED** یا **FAILED** قرار دارد. مؤلفه‌ها به دو دسته تقسیم می‌شوند:

- ماژول:** کوچک‌ترین واحد از فرآیند نصب است که زیرمجموعه‌ای ندارد. با فراخوانی این مؤلفه، وضعیت آن مستقیماً به **INSTALLED** تغییر می‌یابد. این کلاس باید قابلیت بازگردانی وضعیت خود (تغییر مجدد وضعیت به حالت **PENDING**) را نیز داشته باشد.
- پکیج:** مؤلفه‌ای که می‌تواند شامل چندین ماژول یا پکیج دیگر به عنوان پیش‌نیاز باشد و آن‌ها را به ترتیب اجرا می‌کند. اگر اجرای یکی از زیرمجموعه‌ها با خطا مواجه شود، این کلاس موظف است متد **عقب‌گرد** (Rollback) را فقط روی آن دسته از زیرمجموعه‌هایی که در جریان همین تراکنش فعلی اجرا و نصب شده‌اند فراخوانی کرده و وضعیت خود را به **FAILED** تغییر دهد.

توجه

تضمین می‌شود که ساختار وابستگی مؤلفه‌ها هیچ‌گاه تشکیل حلقه نمی‌دهد. به عبارتی، یک مؤلفه می‌تواند همزمان زیرمجموعه چند پکیج مختلف باشد، اما هیچ‌گاه یک مؤلفه به صورت مستقیم یا غیرمستقیم پیش‌نیاز خودش نخواهد شد.

- موتور نصب:** کلاس مرکزی و نقطه شروع سیستم است. این کلاس تمامی مؤلفه‌های ایجاد شده را در یک لیست جامع در حافظه خود نگهداری و مدیریت می‌کند.

راهنمایی: مالکیت حافظه در C++

موتور نصب مسئولیت آزادسازی صحیح حافظه برای جلوگیری از نشت حافظه را بر عهده دارد. با توجه به اینکه اشیاء با عملگر **new** در حافظه پویا ساخته می‌شوند، باید با عملگر **delete** پاک شوند. برای جلوگیری از خطای **Double Free**، کلاس موتور نصب باید تنها مالک اصلی تمام اشیاء باشد. سایر کلاس‌ها (مثل پکیج‌ها) صرفاً از اشاره‌گرها برای دسترسی استفاده می‌کنند و حق آزادسازی حافظه فرزندان خود را ندارند.



۲. مشخصات ورودی

برنامه‌ی شما باید دستورات را خط به خط از ورودی استاندارد بخواند و آن‌ها را تجزیه کند. ساختار دستورات با یکدیگر متفاوت است و بسته به نوع هر دستور، ممکن است هیچ، یک یا چند آرگومان در مقابل آن قرار گیرد. در تجزیه‌ی (parse) این بخش‌ها، قوانین کلی زیر برقرار است:

- **شناسه‌ها (ID):** در صورت وجود در یک دستور، همواره رشته‌هایی پیوسته و بدون فاصله هستند.
- **عنوان (Title):** اگر دستوری دارای عنوان باشد، این عنوان شامل تمامی کاراکترهای باقی‌مانده تا انتهای همان خط است.
- **پایان برنامه:** پس از وارد شدن تمامی دستورات، کلمه‌ی کلیدی `END` به تنهایی و در یک خط مجزا قرار می‌گیرد که نشان‌دهنده‌ی پایان ورودی‌ها و بسته شدن برنامه است.

۱. ایجاد مؤلفه جدید:

- `ADD MODULE <ID> <Title>` – یک شیء جدید از نوع ماژول در حافظه ایجاد می‌کند.
- `ADD PACKAGE <ID> <Title>` – یک شیء جدید از نوع پکیج در حافظه ایجاد می‌کند.

۲. افزودن پیش‌نیاز:

- `ATTACH <Package_ID> <Child_ID>` – مؤلفه `<Child_ID>` را به عنوان یک پیش‌نیاز به پکیج `<Package_ID>` اضافه می‌کند. توجه کنید که در فرآیند نصب، پیش‌نیازها دقیقاً به همان ترتیبی که به والد خود افزوده شده‌اند اجرا خواهند شد.

⚠ توجه

افزودن نیازمندی جدید به پکیجی که از قبل با موفقیت نصب شده (`INSTALLED`) مجاز نیست.

۳. شبیه‌سازی خطا:

- `MOCK_FAIL <ID>` – این دستور صرفاً یک پرچم خطای عمدی را روی مؤلفه فعال می‌کند. در زمان نصب، اگر سیستم به مؤلفه‌ای برسد که این پرچم در آن فعال است، نباید وارد منطق داخلی یا لیست فرزندان آن شود و وضعیت آن مؤلفه فوراً به `FAILED` تغییر می‌کند.

⚠ توجه

اعمال این دستور روی مؤلفه‌ای که از قبل نصب شده (`INSTALLED`) مجاز نیست.

۴. رفع خطای شبیه‌سازی شده:

- `RESOLVE_FAIL <ID>` – پرچم خطای عمدی (`MOCK_FAIL`) را از روی مؤلفه مشخص‌شده غیرفعال می‌کند.



۵. شروع فرآیند نصب:

- `INSTALL <ID>` – مؤلفه مشخص شده و تمامی پیش‌نیازهای آن را نصب می‌کند. مؤلفه‌ای که مستقیماً در این دستور فراخوانی شود، پرچم نصب مستقیم آن روشن می‌شود.

در صورت بروز خطا در فرآیند نصب پیش‌نیازها، وضعیت‌ها به شکل زیر تغییر می‌کنند:

- **تغییر وضعیت به `FAILED`**: مؤلفه‌ی خطا دار، پکیجی که دستور نصب مستقیماً روی آن اجرا شده است، و تمامی پکیج‌های میانی در این زنجیره (مسیر بین پکیج اصلی و مؤلفه خطا دار) به حالت `FAILED` درمی‌آیند.

- **عملیات عقب‌گرد به `PENDING`**: عملیات عقب‌گرد فقط روی پیش‌نیازهایی اعمال می‌شود که در جریان اجرای همین دستور فعلی با موفقیت نصب شده بودند. این مؤلفه‌ها باید به ترتیب معکوس (LIFO) نسبت به زمان نصبشان، به حالت `PENDING` بازگردند.

- **مؤلفه‌های بدون تغییر:** مؤلفه‌هایی که پیش از اجرای این دستور و به هر نحو دیگری نصب شده بودند، تحت تأثیر عملیات عقب‌گرد قرار نمی‌گیرند و `INSTALLED` باقی می‌مانند.

راهنمایی: تلاش مجدد برای نصب

دقت کنید که وضعیت `FAILED` یک وضعیت غیرقابل بازگشت نیست! برای درک بهتر، این سناریو را در نظر بگیرید: فرض کنید پکیج C2 به پکیج C1 وابسته است. در حین اجرای دستور `INSTALL C2`، نصب C1 به دلیل خطای یکی از ماژول‌های زیرمجموعه‌اش شکست می‌خورد. در نتیجه، عملیات متوقف شده و وضعیت هر دو پکیج C1 و C2 برابر `FAILED` می‌شود. پس از اینکه کاربر خطای آن ماژول درونی را برطرف کرد، لزوماً نباید به خاطر داشته باشد که ابتدا C1 را به صورت دستی نصب کند! او صرفاً دوباره دستور `INSTALL C2` را می‌زند و انتظار دارد سیستم به صورت خودکار به سراغ C1 رفته و نصب را تکمیل کند. بنابراین، مراقب باشید که وضعیت `FAILED` را به عنوان یک شرط مسدودکننده بررسی نکنید. اگر متد نصب روی مؤلفه‌ای در وضعیت `FAILED` فراخوانی شد، آن مؤلفه موظف است مجدداً شانس خود را امتحان کرده و فرآیند نصب فرزندان را تکرار کند. اما یک استثنای حیاتی وجود دارد: پرچم `MOCK_FAIL` یک بن‌بست بی‌چون‌وچرا است! در این حالت سیستم باید فوراً متوقف شود.

راهنمایی: مدیریت و نشت وابستگی

اگر سیستم در جریان نصب یک پکیج، به پیش‌نیازی رسید که قبلاً `INSTALLED` شده است، باید بدون اجرای مجدد از آن عبور کند، اما موظف است شمارنده‌ی وابستگی (تعداد والد‌هایی که به آن نیاز دارند) را یک واحد افزایش دهد تا ثبت شود که والد جدیدی هم به آن وابسته شده است.

همچنین در صورت شکست عملیات نصب و فعال شدن فرآیند عقب‌گرد، باید حواستان به این شمارنده‌ها باشد؛ شمارنده‌ی وابستگی مؤلفه‌هایی که صرفاً در همین دستور جاری بالا رفته بود نیز باید (به ترتیب معکوس) یک واحد کاهش یابد تا وابستگی خیالی در سیستم باقی نماند.

برای مدیریت صحیح این منطق، توصیه می‌شود دو متغیر کاملاً مجزا در نظر بگیرید: یک متغیر برای شمارش تعداد والد‌ها و یک پرچم جداگانه برای مشخص کردن وضعیت نصب مستقیم. مراقب باشید که این دو مفهوم را با هم ترکیب نکنید!



۶. حذف نصب: این دستور دارای دو حالت زیر است:

- `UNINSTALL <ID>` – این دستور یک مؤلفه را که قبلاً نصب شده است، لغو نصب می‌کند. با اجرای این دستور، ابتدا وضعیت خود مؤلفه‌ی مشخص‌شده به حالت `PENDING` بازمی‌گردد. سپس، متد حذف نصب روی آن دسته از پیش‌نیازهایی فراخوانی می‌شود که به جز مؤلفه‌ای که در حال حذف است، هیچ پکیج والد دیگری به آن‌ها وابسته نباشد (یعنی با حذف این والد، تعداد والد‌های وابسته به آن‌ها به صفر برسد) و خودشان نیز مستقیماً نصب نشده باشند. این عملیات به ترتیب معکوس افزوده‌شدن پیش‌نیازها انجام می‌شود.
- `UNINSTALL -A` – برای پاکسازی کامل سیستم استفاده می‌شود. موتور نصب روی لیست جامع تمام مؤلفه‌های موجود از آخرین مؤلفه‌ی ایجاد شده به سمت اولین مؤلفه (پیمایش معکوس) حرکت می‌کند و وضعیت هر مؤلفه‌ای که `INSTALLED` یا `FAILED` باشد را به همراه صفر کردن وابستگی‌ها و پرچم‌ها به حالت `PENDING` برمی‌گرداند.

مثال ورودی

```
ADD MODULE S1 Check OS Version
ADD MODULE S2 Download Core Binary
ADD MODULE S3 Extract Core Binary
ADD MODULE S4 Configure Firewall Rules
ADD MODULE S5 Create Admin User
ADD PACKAGE C1 Install Core Engine
ADD PACKAGE C2 Base System Setup
ADD PACKAGE C3 Full App Deployment
ATTACH C1 S2
ATTACH C1 S3
ATTACH C2 S1
ATTACH C2 C1
ATTACH C2 S4
ATTACH C3 C2
ATTACH C3 S5
INSTALL S1
MOCK_FAIL S4
INSTALL C3
RESOLVE_FAIL S4
INSTALL C3
INVALID_CMD C3
INSTALL C1
UNINSTALL C3
UNINSTALL S1
UNINSTALL -A
END
```



۳. مشخصات خروجی

چاپ خروجی‌ها در این سیستم باید صرفاً بر اساس تغییر وضعیت مؤلفه‌ها انجام شود. برای پیاده‌سازی این بخش باید از الگوی طراحی Observer استفاده کنید؛ به این شکل که یک کلاس گزارش‌گیر (مثل `SystemLogger`) را به عنوان ناظر در نظر بگیرید تا هر زمان که وضعیت یک پکیج یا ماژول تغییر کرد، بلافاصله متوجه این تغییر شده و پیام مناسب را در خروجی چاپ کند.

📌 تعریف: الگوی طراحی Observer

الگوی طراحی Observer (ناظر) راهکاری کاربردی برای زمانی است که می‌خواهیم تغییرات یک بخش از برنامه را به بخش‌های دیگر اطلاع دهیم، بدون اینکه کدهای آن‌ها را به هم گره بزنیم. این الگو شبیه یک سیستم اشتراک یا خبرنامه عمل می‌کند؛ یک شیء (به عنوان منبع خبر) تغییرات خود را اعلام می‌کند و سایر اشیائی که به عنوان «ناظر» ثبت‌نام کرده‌اند، این خبر را دریافت کرده و واکنش نشان می‌دهند.

دلیل استفاده از این الگو در این تمرین، رعایت اصل تفکیک وظایف است. کلاس‌های مؤلفه (مثل `Installable`) نباید درگیر جزئیات چاپ خروجی (مانند استفاده مستقیم از `cout`) شوند. وظیفه اصلی آن‌ها فقط انجام عملیات نصب یا حذف و سپس «اعلام» تغییر وضعیتشان است. در سمت دیگر، یک سیستم لاگ‌گیری که نقش «ناظر» را بازی می‌کند، این اعلام‌ها را می‌شنود و زحمت چاپ پیام مناسب در کنسول را بر عهده می‌گیرد.

📌 راهنمایی: پیاده‌سازی الگوی Observer

- **دروازه‌ی واحد تغییر وضعیت** (`setState`): به جای پراکندن کدهای اطلاع‌رسانی در کل برنامه، یک متد `public` به نام `setState` در کلاس پایه (`Installable`) بسازید. تمام تغییر وضعیت‌ها باید فقط از این دروازه عبور کنند. این متد چک می‌کند که آیا وضعیت واقعاً تغییر کرده است؟ اگر بله، وضعیت را به‌روز کرده و به صورت خودکار ناظر را خبر می‌کند.
- **زمان‌بندی اتصال ناظر:** وضعیت اولیه مؤلفه‌ها `PENDING` است و این مقداردهی اولیه نباید لاگ شود. بنابراین بهترین کار این است که در موتور نصب (`InstallationEngine`)، دقیقاً پس از ساختن هر شیء جدید با `new`، لاگر سیستم را از طریق متدی مثل `addObserver` به آن متصل کنید.
- **جادوی عقب‌گرد خودکار:** مزیت بزرگ تمرکز منطق در متد `setState` این است که در زمان عقب‌گرد، نیازی به نوشتن هیچ کد اضافه‌ای برای لاگ‌گیری ندارید! برگرداندن وضعیت مؤلفه‌ها به `PENDING`، خودبه‌خود باعث ارسال خبر به ناظر و چاپ پیام درست در خروجی می‌شود.

خروجی‌های سیستم باید مطابق فرمت‌های زیر باشند:

۱. **لاگ‌های تغییر وضعیت:** هر زمان که وضعیت یک مؤلفه تغییر کند، باید بلافاصله پیام زیر چاپ شود:

• `[OBSERVER] Component <ID> changed from <OLD_STATE> to <NEW_STATE>`

توجه داشته باشید که در زمان ایجاد یک مؤلفه، وضعیت اولیه آن `PENDING` است، اما این مقداردهی اولیه نباید رویداد تغییر وضعیتی در خروجی چاپ کند.



۲. خطاهای اعتبارسنجی و دستورات: در صورت بروز خطا، پیام متناظر با پیشوند `ERROR:` چاپ شده و سیستم منتظر دستور بعدی می‌ماند. در صورت وجود چند خطای همزمان، سیستم تنها اولین خطای پیدا شده را بر اساس اولویت زیر چاپ می‌کند:

- اولویت ۱ (سینتکس نامعتبر یا کمبود آرگومان‌ها):

`ERROR: Invalid command`

- اولویت ۲ (عدم وجود شناسه در سیستم):

`ERROR: Component <ID> does not exist`

⚠ توجه

در دستور `ATTACH` (که تنها دستور دو شناسه‌ای سیستم است)، بررسی وجود شناسه‌ها در سیستم باید از چپ به راست انجام شود. به این معنی که اگر هیچ‌یک از دو شناسه موجود نباشند، فقط خطای مربوط به شناسه اول (سمت چپ) چاپ می‌شود. همچنین، سیستم تنها در صورتی مجاز است به سراغ بررسی خطاهای دیگر (مانند اولویت ۳) برود که وجود هر دو شناسه تأیید شده باشد.

- اولویت ۳ (خطای سازگاری انواع):

`ATTACH` روی ماژول: `ERROR: Cannot attach to a module`

- اولویت ۴ (تکراری بودن ساختار):

شناسه تکراری در `ADD`: `ERROR: Component with ID <ID> already exists`

اتصال تکراری در `ATTACH`: `ERROR: Component <Child_ID> is already attached to <Parent_ID>`

خطای مجدد در `MOCK_FAIL`: `ERROR: Component <ID> is already set to fail`

- اولویت ۵ (خطاهای مربوط به وضعیت فعلی در زمان اجرا):

`INSTALL` یا `MOCK_FAIL` روی مؤلفه‌ای که از قبل نصب شده است:

`ERROR: Component <ID> is already installed`

`ATTACH` به پکیجی که از قبل نصب شده است: `ERROR: Cannot attach to an already installed package`

`RESOLVE_FAIL` روی مؤلفه‌ای که در وضعیت `FAILED` قرار ندارد:

`ERROR: Component <ID> is not in a mock fail state`

`UNINSTALL` مؤلفه‌ای که نصب نیست: `ERROR: Component <ID> is not currently installed`

`UNINSTALL` مؤلفه‌ای که وابستگی والد دیگری است:

`ERROR: Component <ID> is required by another package`

اجرای `UNINSTALL -A` وقتی هیچ چیز نصب نیست: `ERROR: No installed components to uninstall`



⚠ توجه

خطای «وابستگی به والد» منحصراً هنگام اجرای مستقیم دستور `UNINSTALL` چاپ می‌شود. در فرآیند حذف خودکار پیش‌نیازها، اگر سیستم به مؤلفه‌ای برسد که همچنان مورد نیاز پکیج دیگری باشد، بدون چاپ هیچ خطایی، صرفاً از حذف آن مؤلفه چشم‌پوشی کرده و به بررسی سایر پیش‌نیازها ادامه می‌دهد.

مثال خروجی

```
[OBSERVER] Component S1 changed from PENDING to INSTALLED
[OBSERVER] Component S2 changed from PENDING to INSTALLED
[OBSERVER] Component S3 changed from PENDING to INSTALLED
[OBSERVER] Component C1 changed from PENDING to INSTALLED
[OBSERVER] Component S4 changed from PENDING to FAILED
[OBSERVER] Component C1 changed from INSTALLED to PENDING
[OBSERVER] Component S3 changed from INSTALLED to PENDING
[OBSERVER] Component S2 changed from INSTALLED to PENDING
[OBSERVER] Component C2 changed from PENDING to FAILED
[OBSERVER] Component C3 changed from PENDING to FAILED
[OBSERVER] Component S2 changed from PENDING to INSTALLED
[OBSERVER] Component S3 changed from PENDING to INSTALLED
[OBSERVER] Component C1 changed from PENDING to INSTALLED
[OBSERVER] Component S4 changed from FAILED to INSTALLED
[OBSERVER] Component C2 changed from FAILED to INSTALLED
[OBSERVER] Component S5 changed from PENDING to INSTALLED
[OBSERVER] Component C3 changed from FAILED to INSTALLED
ERROR: Invalid command
ERROR: Component C1 is already installed
[OBSERVER] Component C3 changed from INSTALLED to PENDING
[OBSERVER] Component S5 changed from INSTALLED to PENDING
[OBSERVER] Component C2 changed from INSTALLED to PENDING
[OBSERVER] Component S4 changed from INSTALLED to PENDING
[OBSERVER] Component C1 changed from INSTALLED to PENDING
[OBSERVER] Component S3 changed from INSTALLED to PENDING
[OBSERVER] Component S2 changed from INSTALLED to PENDING
[OBSERVER] Component S1 changed from INSTALLED to PENDING
ERROR: No installed components to uninstall
```



۴. الزامات، نکات پیاده‌سازی و تحویل

برای کسب نمره کامل این تمرین، رعایت موارد زیر الزامی است:

۱. رعایت اصول شی‌گرایی: تمامی منطق برنامه باید در قالب کلاس‌ها پیاده‌سازی شود. متغیرهای کلاس‌ها باید دارای سطح دسترسی مناسب (private یا protected) باشند. نوشتن کدهای طولانی و رویه‌ای در تابع main نمره منفی خواهد داشت.
۲. مدیریت حافظه و جلوگیری از نشت حافظه: موتور نصب باید تنها کلاسی باشد که اشیاء را تخصیص داده و در مخرب خود پاک می‌کند. هیچ‌گونه نشت حافظه‌ای در پایان برنامه نباید وجود داشته باشد.
۳. ساختار چندفایلی (Multi-file): پروژه شما باید حتماً به صورت چندفایلی پیاده‌سازی و ساختاردهی شود.
۴. نام‌گذاری و فرمت تحویل: نام فایل پروژه شما باید به فرمت "Name"_StudentID"_CA3 باشد (برای مثال: SoroushNasiri_810803098_CA3).
۵. تنظیمات Makefile: نام فایل makefile شما باید دقیقاً Makefile باشد و فایل اجرایی نهایی پس از انجام عمل make باید با نام installer تولید شود.
۶. ساختار دایرکتوری پروژه: ساختار پروژه باید به گونه‌ای باشد که فایل installer و Makefile حتماً در ریشه (root) پروژه، در کنار فایل localJudge.sh، فایل PDF تمرین و پوشه‌های تست‌کیس (مطابق فایل پایه‌ای که ارائه شده است) قرار گیرند.
۷. سیستم داوری محلی: عملکرد کد شما توسط سیستم داوری محلی که در فایل localJudge.sh قرار دارد بررسی خواهد شد. برای اجرای این سیستم از دستورات زیر استفاده کنید:

```
chmod +x localJudge.sh
./localJudge.sh
```

۵. ساختار کد پیشنهادی

اسکلت کلی زیر جهت پیاده‌سازی کلاس‌ها پیشنهاد می‌شود. دقت کنید که اگرچه این ساختار اولیه برای سادگی در قالب یک فایل نمایش داده شده است، اما شما الزاماً باید پروژه خود را به صورت چند فایلی (Multi-file) پیاده‌سازی کرده و متدها را تکمیل کنید:

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

// --- Status Enumeration ---
enum class ComponentState {
    PENDING,
    INSTALLED,
    FAILED
};
```



```
// --- Suggested Helper Functions ---
string getNextToken(string& str) { /* ... */ }
string trimLeading(const string& str) { /* ... */ }
string stateToString(ComponentState s); // used by logger

// --- Forward Declarations ---
class Installable;

// --- Transaction Context ---
struct TransactionContext {
    vector<Installable*> stateChangedNodes;
    vector<Installable*> countIncreasedNodes;
};

// --- Observer Interface ---
class Observer {
public:
    virtual void onStateChanged(const Installable* comp,
        ComponentState oldState, ComponentState newState) = 0;
    virtual ~Observer() {}
};

// --- Base Class ---
class Installable {
protected:
    string id;
    string title;
    ComponentState state;
    bool mockFail;
    int installedParentsCount;
    bool isExplicitlyInstalled;
    vector<Observer*> observers;
public:
    Installable(string id, string title);
    virtual ~Installable() {}
    void setState(ComponentState newState);
    string getId() const { return id; }
    string getTitle() const { return title; }
    ComponentState getState() const { return state; }
    bool isMockFail() const;
    void setMockFail(bool val);
    int getInstalledParentsCount() const;
    void incParents();
    void decParents();
    bool getExplicit() const;
    void setExplicit(bool val);
    void addObserver(Observer* obs);
    virtual bool isPackage() const { return false; }
    virtual bool install(TransactionContext& tx) = 0;
    virtual void uninstall() = 0;
    virtual void forcePending();
};
```



```
// --- Concrete Observer ---
class SystemLogger : public Observer {
public:
    void onStateChanged(const Installable* comp,
        ComponentState oldState, ComponentState newState) override;
};

// --- Module (Leaf) ---
class Module : public Installable {
public:
    Module(string id, string title);
    bool install(TransactionContext& tx) override;
    void uninstall() override;
};

// --- Package (Composite) ---
class Package : public Installable {
private:
    vector<Installable*> children;
public:
    Package(string id, string title);
    bool isPackage() const override { return true; }
    void addChild(Installable* child);
    bool hasChild(const string& childId) const;
    bool install(TransactionContext& tx) override;
    void uninstall() override;
};

// --- Installation Engine ---
class InstallationEngine {
private:
    vector<Installable*> allComponents;
    SystemLogger logger;
    Installable* getComponent(const string& id);
public:
    InstallationEngine();
    ~InstallationEngine(); // Must delete all allocated components
    void processCommand(string line);
};
```